

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Infrastructure as Code with Ansible

Unit 13 · Linux (10-151-105)



Learning Outcomes

This unit applies two course competencies — and adds version control as a secondary focus.

Competency	In this unit you will...
C2 — Command-line administration	Run ad-hoc commands and playbooks, author inventory and YAML, and version-control your work with Git.
C6 — Manage applications	Install and manage package state declaratively across Rocky (dnf) and Ubuntu (apt) from one task.
Secondary — Version control	Use Git to track changes, branch and merge, work with remotes, and keep secrets out of the repo.

Outcome: bring two hosts to a desired state with a single, re-runnable playbook — and manage that playbook in Git.

LESSON 1

From Scripting to Desired State



Automation vs. Orchestration

Automation

Configure the system to perform a single task for you — a backup script, a package install.

Key attribute: one task is completed.

Orchestration

Coordinate many automated tasks into one workflow — provision a VM, install a DB, open the firewall, hand it off.

Key attribute: a series of tasks runs as one event.

The terms are not interchangeable.

Why it matters to a Linux admin:

Doing identical work by hand across many servers is slow and inconsistent — a skipped step produces configuration drift, where machines that should match quietly don't. Automation and orchestration remove that drift.

Infrastructure as Code (IaC)

Manage systems through version-controlled configuration files rather than by hand. You declare the desired configuration in text, store it under version control, and let a tool make every target match it. Because Linux is configured through text files, it is ideally suited to IaC.

Consistency

Every target gets the same settings — no drift.

Speed

One change rolls out everywhere at once.

Recoverability

The file is the documentation, and it's versioned.

GitOps takes this further — Git becomes the single source of truth that drives configuration.

Declarative vs. Imperative

Style	You write...	Example
Imperative	The exact steps and commands to reach the result.	A Bash script: useradd, then mkdir, then chmod — in order, every time.
Declarative	The end state you want; the tool figures out the steps.	"User kgarcia exists in group devs." The tool acts only if it isn't already true.

Where does Ansible fall? Each task declares a desired end state and is idempotent (declarative), but the playbook runs its tasks in the order you write them (procedural). More declarative than a script; less purely declarative than Puppet.

LESSON 2

Ansible



Meet Ansible

An agentless, SSH-based configuration manager that brings systems to a desired state defined in YAML.



Agentless

No software installed on managed hosts — Ansible connects over ordinary SSH. Fewer moving parts.



Speaks YAML

Playbooks are readable enough to double as documentation of how a system is configured.

Control node — the Linux host running Ansible (your Rocky VM). **Managed nodes** — the targets it configures (Rocky itself + Ubuntu); they need no Ansible, only SSH reachability.

Why keys matter: *key-based SSH lets Ansible connect non-interactively — no password prompt stops the automation. (You set this up in Unit 10.)*

The Inventory

The inventory is the list of hosts Ansible manages — optionally collected into named groups so one playbook can target them together.

```
# inventory.ini — the two hosts in your pod
[lab]
rocky10.lab.local    ansible_connection=local
ubuntu24.lab.local  ansible_user=student
```

The `[lab]` group holds both hosts. Rocky talks to itself with a `local` connection; Ubuntu is reached over SSH as `student`.

Ad-hoc Commands

A single task fired at the inventory, no playbook required — great for quick checks and one-offs. Form: `ansible <pattern> -m <module> -a "<args>"`

```
# Are both hosts reachable? (ping module checks connectivity, not ICMP)
```

```
ansible lab -m ping
```

```
# Show uptime on every host in the inventory
```

```
ansible lab -m command -a "uptime"
```

```
# Ensure the 'tree' package is installed on both
```

```
ansible lab -m package -a "name=tree state=present" --become
```

Ad-hoc commands are great for exploring; the real strength is capturing repeatable work in playbooks.

Modules

Modules are the predefined units of work that act on managed nodes — thousands exist. A few you'll use:

Module	What it manages
<code>package</code>	Software packages using the host's native manager — dnf on Rocky, apt on Ubuntu — from one task.
<code>service</code>	Service state: started/stopped, enabled/disabled at boot.
<code>user</code>	User accounts — create, modify, set group membership.
<code>copy / file</code>	Placing files; setting ownership, permissions, and directory state.

Dual-distro payoff: one task — `name=tree state=present` — installs on a dnf host and an apt host alike. You declare what; Ansible handles how on each distro.

Playbooks

A YAML file of plays (host targets) and tasks (desired-state steps). This one configures both hosts:

```
# site.yml
---
- name: Configure lab hosts
  hosts: lab
  become: true
  tasks:
    - name: Ensure 'tree' is installed
      package:
        name: tree
        state: present

    - name: Ensure cron is enabled
      service:
        name: "{{ 'crond' if ansible_os_family
                  == 'RedHat' else 'cron' }}"
        state: started
        enabled: true
```

Facts adapt it to each host.

Before running, Ansible gathers facts about each node — OS family, hostname, IP. Here `ansible_os_family` picks the right service name (crond vs. cron), so one playbook fits Rocky and Ubuntu.

Run it with:

```
ansible-playbook site.yml
```

Idempotency — the Whole Point

Run the playbook once: each task reports **changed**. Run it again: every task reports **ok** — nothing changes, because the system already matches the desired state.

```
# Second run — nothing left to do
PLAY RECAP *****
rocky10.lab.local    : ok=4    changed=0    unreachable=0    failed=0
ubuntu24.lab.local  : ok=4    changed=0    unreachable=0    failed=0
```

changed=0 on a re-run is the signature of declarative, idempotent configuration — exactly what a Bash script can't promise. It's why a baseline is safe to re-apply on a schedule: it costs nothing when the system complies, and quietly corrects drift when it doesn't.

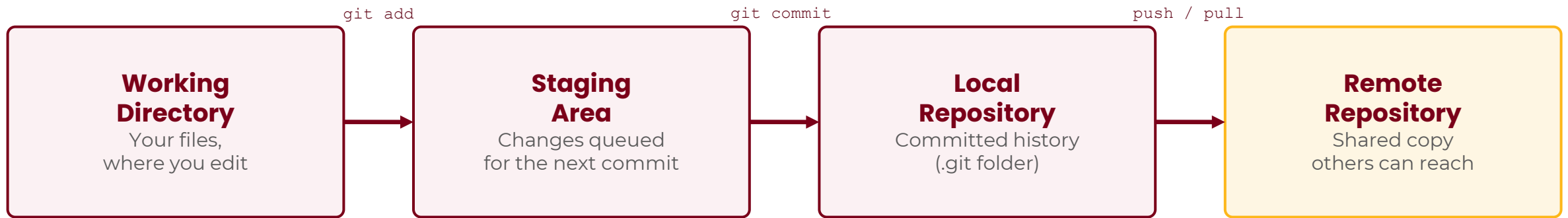
LESSON 3

Version Control with Git



Git: The Three-Tree Model

Your work moves through three local areas (four, once a remote is involved). Understanding this is understanding Git.



You edit in the working directory, `git add` the changes you want to record into staging, then `git commit` them into the repository as a permanent snapshot. Staging is what lets you commit some edits and not others.

The Everyday Git Workflow

```
# One-time identity (stamped on every commit)
git config --global user.name "Student Name"
git config --global user.email "student@mstc.edu"

# Start a repository
mkdir ~/ansible-lab && cd ~/ansible-lab
git init

# Stage and commit a snapshot
git status          # what's changed?
git add site.yml    # stage this file
git commit -m "Add lab playbook"

# See changes, then review history
git diff
git log --oneline
```

The loop never changes:

edit →

`git add` → `git commit`

with

`status`, `diff`, and `log` to see where you stand. `git checkout` returns you to an earlier commit if a change goes wrong.

Branching, Remotes & .gitignore

Branching

```
git branch add-firewall
git checkout add-firewall
# ...commit changes...
git checkout main
git merge add-firewall
```

Work on a change safely off main; merge it back when ready, discard it if not.

.gitignore — keep secrets out

```
*.log
vault-password.txt # never commit secrets
*.key
```

Remotes

`git clone` — copy a remote repo locally
`git pull` — fetch + merge others' changes
`git push` — upload your commits to share

In your offline pod: Git stays local; a bare repo on one host can serve the other over SSH — same commands, no internet.

A credential committed to a repo lives in its history permanently — even after you delete the file. Add sensitive files to .gitignore before the first commit.

LESSON 4

The Wider Landscape



Puppet — Continuous Enforcement

Specializes in keeping large fleets in a defined state over time.

Uses a master-agent model: an agent on each host checks in regularly and continuously enforces the desired configuration, self-correcting drift between runs. Fully declarative, written in a Ruby-like DSL stored in manifests; master and agents authenticate with X.509 certificates from Puppet's own CA.

```
# Puppet manifest (.pp)
package { 'tree':
  ensure => installed,
}

service { 'crond':
  ensure => running,
  enable => true,
}
```

Reach for it when: you need continuous, self-correcting enforcement across a large, long-lived fleet.

OpenTofu — Provisioning

Specializes in provisioning infrastructure, not configuring software inside it.

A community-driven, open-source fork of Terraform. It creates VMs, networks, cloud storage, load balancers, and DNS records. Declarative; uses providers (plugins) to talk to each platform and tracks the current state of your infrastructure in a file so it knows what exists and what to change.

```
# OpenTofu config (.tf)
resource "local_file" "ex" {
  filename = "/tmp/hello.txt"
  content  = "Managed by OpenTofu"
}
```

Reach for it when: you're standing up or tearing down infrastructure — especially cloud resources — with a tracked state of what exists.

Telling the Tools Apart

Tool	Model	Specializes in
Ansible	Agentless, runs on demand over SSH	Configuring software on hosts; quick, on-demand runs
Puppet	Agent-based, continuous enforcement	Keeping large fleets in a defined state over time
OpenTofu	Declarative, state-tracked provisioning	Building infrastructure — VMs, networks, cloud resources

OpenTofu builds the house (provisions infrastructure); Ansible and Puppet furnish it (configure what runs inside). Many shops use one of each.

The Lab



NETLAB+ Lab: Infrastructure as Code with Ansible

- 1 Configure SSH-based Ansible access from the Rocky control node to both hosts.
- 2 Run ad-hoc commands to confirm reachability and gather facts.
- 3 Complete a partially-written playbook (hybrid) — add tasks, then run it across Rocky and Ubuntu.
- 4 Re-run the playbook to confirm idempotency (changed=0).
- 5 Place the playbook under Git version control and practice the commit workflow.

Security Focus



Automation Multiplies Everything — Including Mistakes

Never hard-code credentials in a playbook — encrypt secrets with Ansible Vault.

Never commit a secret to Git — repo history is permanent; exclude key/vault files with .gitignore.

Give automation its own least-privilege account — not blanket root everywhere.

Treat idempotent enforcement as a control — a baseline you can re-apply and verify.

Lean on the audit trail — declarative config + Git history = who changed what, when, and why.

Why it matters: a manual mistake harms one host; an automated mistake harms every host at once.

Key Takeaways

- ✓ IaC means declaring desired state in version-controlled files — consistent, fast, recoverable.
- ✓ Ansible is agentless (SSH), declarative-per-task, and idempotent — changed=0 on a clean re-run.
- ✓ One playbook with the package/service modules configures both Rocky and Ubuntu via facts.
- ✓ Git tracks your work through working dir → staging → repo; branch, merge, and keep secrets out.
- ✓ Puppet enforces continuously via agents; OpenTofu provisions infrastructure — know what each is for.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 13 · Infrastructure as Code with Ansible

